

**UNIVERSIDADE REGIONAL DO NOROESTE DO ESTADO DO RIO GRANDE DO
SUL
UNIJUÍ
ENGENHARIA DE SOFTWARE**

**BERNARDO ENGELMANN
GUSTAVO CAMPOS
KAUÃ COSTELLA BLASS
PEDRO EMILIO**

**CATHOLYC: PROJETO E IMPLEMENTAÇÃO DE UMA LINGUAGEM DE
PROGRAMAÇÃO TEMÁTICA E DE SEU TRANSPILADOR PARA PYTHON**

Santa Rosa
2026

**BERNARDO ENGELMANN
GUSTAVO CAMPOS
KAUÃ COSTELLA BLASS
PEDRO EMILIO**

**CATHOLYC: PROJETO E IMPLEMENTAÇÃO DE UMA LINGUAGEM DE
PROGRAMAÇÃO TEMÁTICA E DE SEU TRANSPILADOR PARA PYTHON**

Relatório técnico de especificação do Trabalho Prático 1, apresentado à disciplina de Linguagens Formais e Compiladores da Universidade Regional do Noroeste do Estado do Rio Grande do Sul – UNIJUÍ.

Professor: Marcos Ronaldo Melo Cavalheiro.

Santa Rosa
2026

SUMÁRIO

1 INTRODUÇÃO	4
1.1 Objetivos	4
1.2 Organização do relatório	4
2 DECISÕES PRÓPRIAS DE PROJETO	5
3 ESPECIFICAÇÃO LÉXICA	6
3.1 Alfabeto	6
3.2 Espaços em branco, quebras de linha e comentários	6
3.3 Tabela de tokens	6
3.4 Estratégia de maior casamento	7
3.5 Tratamento de erros léxicos	7
4 ESPECIFICAÇÃO SINTÁTICA	8
4.1 Definição formal da gramática	8
4.2 Produções em EBNF	8
4.3 Precedência e ausência de ambiguidade	9
4.4 Análise sintática e erros sintáticos	10
5 ÁRVORE SINTÁTICA ABSTRATA E ANÁLISE SEMÂNTICA	10
5.1 Árvore sintática abstrata	10
5.2 Análise semântica	11
6 GERAÇÃO DE CÓDIGO	12
7 IMPLEMENTAÇÃO E ORGANIZAÇÃO DO CÓDIGO	14
8 SUÍTE DE TESTES	15
9 DECLARAÇÃO DE USO DE FERRAMENTAS DE INTELIGÊNCIA ARTIFICIAL	17
10 CONSIDERAÇÕES FINAIS	17
REFERÊNCIAS	19

1 INTRODUÇÃO

Este relatório documenta o projeto e a implementação da CatHolyc, uma linguagem de programação de tema bíblico e litúrgico, e de seu transpilador (compilador fonte-a-fonte), desenvolvidos como Trabalho Prático 1 da disciplina de Linguagens Formais e Compiladores. A linguagem associa expressões do vocabulário religioso a construções tradicionais de programação, de modo análogo à linguagem esotérica BIRL, adotada como referência de inspiração (BIRL, 2026), mas com sintaxe, gramática e regras semânticas próprias. Os programas têm a extensão `.holyc` e são traduzidos para a linguagem Python.

1.1 Objetivos

O objetivo geral é formalizar o léxico e a sintaxe da linguagem e construir um transpilador capaz de reconhecer programas escritos nela, representá-los por meio de uma Árvore Sintática Abstrata (AST), verificar propriedades semânticas mínimas e gerar código equivalente em Python. Como objetivos específicos, destacam-se:

- a) especificar o alfabeto e os tokens por expressões regulares, com tratamento de erros léxicos;
- b) definir uma Gramática Livre de Contexto $G = (V, T, P, S)$ não ambígua, em notação EBNF, adequada a um analisador preditivo por descida recursiva;
- c) implementar os analisadores léxico, sintático e semântico, separando claramente a validação da forma e a validação do contexto;
- d) gerar código Python a partir da AST, preservando o comportamento do programa de entrada;
- e) oferecer uma interface de linha de comando que transpile e, opcionalmente, execute o programa gerado;
- f) validar o conjunto por uma suíte de testes com casos válidos e inválidos.

1.2 Organização do relatório

A seção 2 apresenta as decisões próprias de projeto. As seções 3 e 4 trazem as especificações léxica e sintática. A seção 5 trata da AST e da análise semântica, e a seção 6, da geração de código. A seção 7 descreve a implementação, a seção 8

apresenta a suíte de testes, a seção 9 contém a declaração de uso de ferramentas de inteligência artificial e a seção 10 reúne as considerações finais.

2 DECISÕES PRÓPRIAS DE PROJETO

Para atender à exigência de originalidade, a CatHolyc não é uma adaptação da BIRL-Lite com troca de palavras-chave. As principais decisões de projeto foram as seguintes:

- a) **Operadores exclusivamente por palavras.** Não existem símbolos operadores na linguagem. Soma, comparação e conectivos lógicos são palavras reservadas (`somado_a`, `conforme_a`, `em_comunhão_com`). Com isso, o problema de prefixo comum entre `=` e `==` desaparece, e o maior casamento passa a incidir sobre palavras, como `amém e amém_senhor`;
- b) **Prefixo `\$` obrigatório nos identificadores**, no estilo da linguagem PHP (`$idade`, `$maioridade`). Uma palavra sem `$` nunca é identificador: se não for reservada, é erro léxico. Isso elimina a ambiguidade entre palavras reservadas e nomes de variáveis, e permite que `$caso` seja um identificador válido;
- c) **Delimitação em dois níveis.** A palavra `amém` encerra comandos e declarações, enquanto `amém_senhor` encerra blocos. As condições de `caso e enquanto` não usam parênteses nem marcador de abertura de bloco: o primeiro comando após a expressão inicia o bloco;
- d) **Repetições sem parênteses.** O laço `peregrine` tem a forma `peregrine <atribuição> <condição> amém <atribuição> <bloco> amém_senhor`, reaproveitando a produção de atribuição para a inicialização e o incremento. O laço `em_cada $v $s` percorre os caracteres de um salmo, pois a linguagem não possui vetores;
- e) **Negação lógica de menor precedência que a comparação.** Em `não_seja $a exaltado_sobre $b`, a negação incide sobre toda a comparação, equivalendo a `not (a > b)`, e não a `(not a) > b`;
- f) **Funções tipadas e controle de laço.** Funções são declaradas com `oficio <tipo> $nome <tipo> $param ... <bloco> amém_senhor`, retornam valor com `retorne` e são chamadas em qualquer expressão

com a sintaxe `$nome(arg, arg)`, usando o mesmo prefixo `$` das variáveis. Os comandos `cessai` e `perseverai` correspondem a `break` e `continue` e só são válidos dentro de laços.

Além disso, o vocabulário original foi estendido com os delimitadores `(, )` e `,,` com `privado_de` também como negação aritmética unária e com dois estilos de comentário descartados pelo léxico: `glosa(...)` e `|> ... <|`.

3 ESPECIFICAÇÃO LÉXICA

3.1 Alfabeto

O alfabeto Σ da linguagem é a união de quatro subconjuntos: $\Sigma = \Sigma_{\text{letras}} \cup \Sigma_{\text{dígitos}} \cup \Sigma_{\text{especiais}} \cup \Sigma_{\text{brancos}}$. O subconjunto Σ_{letras} contém as letras de `a` a `z` e de `A` a `Z`, além das letras acentuadas do português; $\Sigma_{\text{dígitos}}$ contém os algarismos de 0 a 9; $\Sigma_{\text{especiais}}$ contém `_ + - * / % = > < ! & | ( ) " . , $;` e Σ_{brancos} contém o espaço, a tabulação e as quebras de linha (`\n` e `\r\n`). Toda cadeia de entrada é um elemento de Σ^* , e cada categoria de token é uma linguagem regular contida em Σ^* . Qualquer símbolo fora de Σ , fora de literais de texto e de comentários, constitui erro léxico.

3.2 Espaços em branco, quebras de linha e comentários

Espaços, tabulações e quebras de linha pertencem à linguagem regular `[ \t\r\n ]+` e são descartados sem gerar token; a quebra de linha incrementa o contador de linhas, usado na localização dos erros. A linguagem admite dois estilos de comentário, ambos descartados: `glosa(texto)`, reconhecido por `glosa\[ (^ ) ] * \)`, e o comentário em bloco `|> texto <|`, reconhecido de forma não gulosa até o primeiro `<|`, podendo ocupar várias linhas. Um `|>` sem o `<|` correspondente é erro léxico.

3.3 Tabela de tokens

O Quadro 1 reúne as categorias de tokens, suas expressões regulares ou conjuntos de lexemas, exemplos e descrições. As palavras reservadas formam um conjunto finito e fechado.

Quadro 1 – Tabela de tokens da CatHolyc

Categoria	Expressão regular ou lexemas	Exemplo	Descrição
IDENTIFICADOR	\\$[a-zA-Zà-ú_][a-zA-Zà-ú0-9_]*	\$idade	variável ou constante; prefixo \$ obrigatório
NUM_INTEIRO	[0-9]+	18	literal do tipo capítulo
NUM_REAL	[0-9]+\.[0-9]+	10.5	literal do tipo versículo
LITERAL_TEXTO	"([^\n\\] \\.)*"	"Amém\n"	literal do tipo salmo, com escapes
DELIMITADOR	em_nome_do_pai, assim_seja_em_seu_nome_ame m, amém, amém_senhor	amém	início e fim do programa, fim de comando e fim de bloco
TIPO	capítulo, versículo, salmo, dogma	capítulo	tipos int, float, str e bool
DECLARAÇÃO	mandamento, preceito	preceito	constante e variável
ATRIBUIÇÃO	seja	seja	operador de atribuição
OP_ARITMÉTICO	somado_a, privado_de, multiplicado_por, partilhado_entre, díizimo_de	somado_a	soma, subtração, multiplicação, divisão e resto
OP_RELACIONAL	exaltado_sobre, submisso_a, não_abaixo_de, não_acima_de, conforme_a, dissonante_de	conforme_a	maior, menor, maior ou igual, menor ou igual, igual e diferente
OP_LÓGICO	em_comunhão_com, ou_porventura, não_seja	não_seja	conjunção, disjunção e negação
CONTROLE	caso, doutra_sorte_caso, doutra_sorte, enquanto, peregrine, em_cada, cessai, perseverai	caso	decisão, repetição e interrupção de laços
FUNÇÃO	oficio, retorne	oficio	definição de função e retorno de valor
ENTRADA_SAÍDA	proclame, confesse	proclame	saída e entrada de dados
LITERAL_LÓGICO	verdade, falsidade	verdade	valores do tipo dogma
PONTUAÇÃO	() ,	(	agrupamento, chamadas e separação de argumentos
COMENTÁRIO	glosa\[([^\]]*)\] \ >.*?<\	glosa(nota)	descartado
BRANCO	[\t\r\n]+	–	descartado

Fonte: elaborado pelos autores (2026).

3.4 Estratégia de maior casamento

O analisador léxico adota a estratégia de maior casamento (*longest match*). Ao encontrar uma letra ou sublinhado, consome toda a sequência de letras, dígitos e sublinhados e somente depois consulta a tabela de palavras reservadas. Dessa forma, `amém_senhor` não é lido como `amém` seguido de outro lexema, e

`doutra_sorte_caso` não é fragmentado em `doutra_sorte` e `caso`. Como identificadores começam obrigatoriamente por `$`, uma palavra que não é reservada e não tem `$` é rejeitada, e `$caso` é aceito como identificador, e não como a palavra reservada `caso`. O mesmo princípio vale para números: `18` e `18.5` compartilham prefixo, e o literal só é classificado como real quando o ponto é seguido de dígitos; `12.` é rejeitado.

3.5 Tratamento de erros léxicos

O analisador léxico informa a linha e a coluna do lexema inválido e, ao contrário das fases seguintes, não para no primeiro erro: aplica recuperação em modo pânico, retomando a análise no caractere seguinte, e acumula todos os erros encontrados. São erros léxicos: símbolo fora do alfabeto (por exemplo, `@`), palavra sem o prefixo `$`, `$` isolado, número malformado (`12.`), literal de texto não fechado e comentário `|>` sem fechamento. Exemplo de mensagem emitida quando a linha 3 de um programa contém `$x seja $x somado_a @ amém`:

Erro léxico: símbolo ou lexema inválido '@' na linha 3, coluna 21

4 ESPECIFICAÇÃO SINTÁTICA

4.1 Definição formal da gramática

A sintaxe é definida pela Gramática Livre de Contexto $G = (V, T, P, S)$, em que:

- a) $V = \{\text{Programa, Bloco, Comando, DeclConst, DeclVar, Tipo, Atribuicao, ComandoSe, ComandoEnquanto, ComandoPara, ComandoParaCada, ComandoSaida, ComandoEntrada, Cessai, Perseverai, DefFuncao, Retorne, Chamada, ExpressaoOu, ExpressaoE, ExpressaoNao, ExpressaoRel, OpRel, ExpressaoAdd, OpAdd, ExpressaoMul, OpMul, Fator}\};$
- b) $T = \{\text{em_nome_do_pai, assim_seja_em_seu_nome_amem, mandamento, preceito, capítulo, versículo, salmo, dogma, seja, amém, amém_senhor, caso, doutra_sorte_caso, doutra_sorte, enquanto, peregrine, em_cada, cessai, perseverai, oficio, retorne, proclame, confesse, verdade, falsidade,}$

somado_a, privado_de, multiplicado_por, partilhado_entre, dízimo_de, exaltado_sobre, submisso_a, não_abaixo_de, não_acima_de, conforme_a, dissonante_de, em_comunhão_com, ou_porventura, não_seja, (,), ,, IDENT, NUM_INT, NUM_REAL, TEXTO}, em que IDENT, NUM_INT, NUM_REAL e TEXTO são os terminais lexicais definidos por expressões regulares na seção 3;

- c) **S** = Programa;
- d) **P** = conjunto de produções apresentado na subseção 4.2.

4.2 Produções em EBNF

Programa	::= "em_nome_do_pai" Bloco "assim_seja_em_seu_nome_amem"
Bloco	::= { Comando }
Comando	::= DeclConst DeclVar Atribuicao ComandoSe ComandoEnquanto ComandoPara ComandoParaCada ComandoSaida ComandoEntrada Cessai Perseverai DefFuncao Retorne
Tipo	::= "capítulo" "versículo" "salmo" "dogma"
DeclConst	::= "mandamento" Tipo IDENT "seja" ExpressaoOu "amém"
DeclVar	::= "preceito" Tipo IDENT ["seja" ExpressaoOu] "amém"
Atribuicao	::= IDENT "seja" ExpressaoOu "amém"
ComandoSe	::= "caso" ExpressaoOu Bloco { "doutra_sorte_caso" ExpressaoOu Bloco } ["doutra_sorte" Bloco] "amém_senhor"
ComandoEnquanto	::= "enquanto" ExpressaoOu Bloco "amém_senhor"
ComandoPara	::= "peregrine" Atribuicao ExpressaoOu "amém" Atribuicao Bloco "amém_senhor"
ComandoParaCada	::= "em_cada" IDENT IDENT Bloco "amém_senhor"
ComandoSaida	::= "proclame" ExpressaoOu "amém"
ComandoEntrada	::= "confesse" IDENT "amém"
Cessai	::= "cessai" "amém"
Perseverai	::= "perseverai" "amém"
DefFuncao	::= "oficio" Tipo IDENT { Tipo IDENT } Bloco "amém_senhor"
Retorne	::= "retorne" [ExpressaoOu] "amém"
Chamada	::= IDENT "(" [ExpressaoOu { "," ExpressaoOu }] ")"
ExpressaoOu	::= ExpressaoE { "ou_porventura" ExpressaoE }
ExpressaoE	::= ExpressaoNao { "em_comunhão_com" ExpressaoNao }
ExpressaoNao	::= ["não_seja"] ExpressaoRel
ExpressaoRel	::= ExpressaoAdd [OpRel ExpressaoAdd]
OpRel	::= "exaltado_sobre" "submisso_a" "não_abaixo_de" "não_acima_de" "conforme_a" "dissonante_de"
ExpressaoAdd	::= ExpressaoMul { OpAdd ExpressaoMul }
OpAdd	::= "somado_a" "privado_de"
ExpressaoMul	::= Fator { OpMul Fator }
OpMul	::= "multiplicado_por" "partilhado_entre" "dízimo_de"
Fator	::= IDENT NUM_INT NUM_REAL TEXTO "verdade" "falsidade" "(" ExpressaoOu ")" "privado_de" Fator Chamada

Fonte: elaborado pelos autores (2026). Notação EBNF: { } indica repetição e [] indica opcionalidade.

4.3 Precedência e ausência de ambiguidade

A precedência dos operadores está codificada na própria estrutura da gramática, por meio do encadeamento de não terminais de expressão, e não é corrigida posteriormente pelo gerador de código. O Quadro 2 apresenta os níveis, do menor para o maior grau de precedência.

Quadro 2 – Níveis de precedência das expressões

Nível	Não terminal	Operadores	Associatividade
1	ExpressaoOu	ou_porventura	à esquerda
2	ExpressaoE	em_comunhão_com	à esquerda
3	ExpressaoNao	não_seja (prefixo, opcional)	–
4	ExpressaoRel	exaltado_sobre, submisso_a, não_abaixo_de, não_acima_de, conforme_a, dissonante_de	não associativo
5	ExpressaoAdd	somado_a, privado_de	à esquerda
6	ExpressaoMul	multiplicado_por, partilhado_entre, díizimo_de	à esquerda
7	Fator	identificadores, literais, chamadas de função, privado_de (prefixo) e expressões entre parênteses	–

Fonte: elaborado pelos autores (2026).

Como cada nível só combina operandos do nível imediatamente superior, existe uma única árvore de derivação para cada expressão válida. Assim, `2 somado_a 3 multiplicado_por 4` é interpretada como `2 somado_a (3 multiplicado_por 4)`, e `(2 somado_a 3) multiplicado_por 4`, como escrito. A gramática não possui recursão à esquerda: as repetições são expressas por `{ }`, o que permite a implementação por descida recursiva sem retrocesso. A única decisão que olha dois tokens é a de `Fator`: um identificador seguido de `(` é uma chamada de função; caso contrário, é uma variável. A ausência de ambiguidade em caso decorre de o bloco terminar sempre em `doutra_sorte_caso`, `doutra_sorte` ou `amém_senhör`.

4.4 Análise sintática e erros sintáticos

O analisador sintático é um parser preditivo por descida recursiva. Cada não terminal da gramática corresponde a um método da classe `Parser`

(parsePrograma, parseBloco, parseSe, parseExpressaoOu e assim por diante), e a escolha entre alternativas é feita pelo exame do token atual. Ao detectar uma violação, o processamento é interrompido no primeiro erro, e a mensagem informa a linha, o lexema encontrado, sua categoria e o elemento esperado. Por exemplo, para a declaração `preceito capítulo $x seja amém` na linha 2, em que falta a expressão após `seja`, o analisador emite:

Erro sintático: linha 2 – encontrado 'amém' (AMEM), esperado
'expressão (identificador, número, texto, verdade/falsidade, '(')'

5 ÁRVORE SINTÁTICA ABSTRATA E ANÁLISE SEMÂNTICA

5.1 Árvore sintática abstrata

O parser constrói diretamente a AST, com um nó por construção da linguagem: Programa, DeclConst, DeclVar, Atribuicao, Se, Enquanto, Para, ParaCada, Saida, Entrada, Cessai, Perseverai, DefFuncao (com seus Param) e Retorne para os comandos, e OperacaoBinaria, OperacaoUnaria, Identificador, Literal e Chamada para as expressões. Os parênteses não geram nós, pois apenas guiam a análise, e o prefixo \$ é removido na construção do nó Identificador. Cada nó guarda a linha de origem. A Figura 1 mostra a AST do comando `$resultado seja 2 somado_a 3 multiplicado_por 4 amém`, na qual a precedência aparece na estrutura da árvore.

Figura 1 – Estrutura da AST (opção --dump-ast, em JSON, resumida)

```
Atribuicao(nome: resultado)
└─ OperacaoBinaria(somado_a)
    └─ Literal(int, 2)
        └─ OperacaoBinaria(multiplicado_por)
            └─ Literal(int, 3)
                └─ Literal(int, 4)
```

Fonte: elaborado pelos autores (2026).

5.2 Análise semântica

A análise semântica é executada após a sintática e é implementada separadamente dela: a gramática valida a forma do programa, e esta etapa valida propriedades que dependem de contexto. O analisador mantém uma tabela de

símbolos com escopos em pilha, em que cada símbolo guarda nome, tipo, indicação de constante e linha. O escopo global é o corpo do programa, e cada bloco de `caso`, `enquanto`, `peregrine` e `em_cada` empilha um novo escopo, permitindo sombreamento. No `peregrine`, a inicialização e o incremento são atribuições a variáveis já declaradas e ocorrem no escopo envolvente. A variável de iteração de `em_cada` é declarada como salmo, e as funções dividem o espaço de nomes com as variáveis. O analisador interrompe a análise no primeiro erro, e as mensagens recebem o prefixo `ESEM`. As regras verificadas são:

- a) `mandamento` exige valor inicial;
- b) é proibida a redeclaração de um nome no mesmo escopo, e todo identificador deve ser declarado antes do uso;
- c) é proibida a atribuição a um mandamento (constante) e a leitura nele com `confesse`;
- d) em atribuições e inicializações, os tipos devem coincidir, admitindo-se apenas a promoção de capítulo para versículo;
- e) as condições de `caso`, `doutra_sorte_caso`, `enquanto` e `peregrine` devem ser do tipo dogma;
- f) operadores aritméticos exigem operandos numéricos, e o salmo nunca participa de aritmética; o resultado é versículo se algum operando for versículo;
- g) `dízimo_de` exige dois capítulos;
- h) operadores de ordem exigem números, e `conforme_a` e `dissonante_de` aceitam números entre si ou dois operandos do mesmo tipo;
- i) operadores lógicos exigem operandos do tipo dogma;
- j) em `em_cada`, a coleção deve ser um salmo (restrição da versão atual da linguagem);
- k) `cessai` e `perseverai` só podem aparecer dentro de `enquanto`, `peregrine` ou `em_cada`, e uma função aninhada não herda o laço externo;
- l) `retorne` só pode aparecer dentro de `oficio`, e o valor retornado deve ser compatível com o tipo declarado da função; `retorne` sem valor é sempre válido;

- m) uma chamada deve ter como alvo um `oficio` declarado, com o número exato de argumentos e tipos compatíveis com os parâmetros, e seu tipo é o tipo de retorno da função;
- n) dentro de um `oficio`, é proibido escrever (por `seja` ou `confesse`) em variável de escopo externo; nesta versão só é permitida a leitura, pois o Python criaria uma variável local silenciosamente.

6 GERAÇÃO DE CÓDIGO

O gerador de código só é acionado após o sucesso das análises léxica, sintática e semântica e percorre a AST, sem recorrer à substituição textual de palavras-chave. A indentação do Python é controlada por uma pilha: `amém_senhor` apenas desempilha um nível, sem emitir texto, e um bloco vazio gera `pass`. O Quadro 3 resume os principais mapeamentos.

Quadro 3 – Mapeamento das construções da CatHolyc para Python

Construção	Código Python gerado
capítulo, versículo, salmo, dogma	<code>int, float, str, bool</code> (a declaração vira atribuição simples)
mandamento \$x seja E	<code>X = E</code> , com o nome em maiúsculas; a imutabilidade é garantida pela análise semântica
preceito \$x [seja E]	<code>x = E</code> ; sem valor inicial, recebe <code>0, 0.0, ""</code> ou <code>False</code>
somado_a, privado_de, multiplicado_por, dízimo_de	<code>+, -, *, %</code>
partilhado_entre	<code>//</code> se os dois operandos são capítulo; caso contrário, <code>/</code>
operadores relacionais	<code>>, <, >=, <=, ==, !=</code>
em_comunhão_com, ou_porventura, não_seja	<code>and, or, not</code> , com parênteses mínimos pela precedência
verdade, falsidade	<code>True, False</code>
caso, doutra_sorte_caso, doutra_sorte	<code>if, elif, else:</code> com indentação
enquanto	<code>while cond:</code>
peregrine	inicialização, depois <code>while cond:</code> com o corpo e o incremento ao final
em_cada \$v \$s	<code>for v in s:</code>
cessai, perseverai	<code>break, continue</code>
oficio T \$f T \$p ...	<code>def f(p, ...):</code> com o corpo indentado; pode ser aninhado
retorne E, retorne	<code>return E, return</code>
\$f(a, b)	<code>f(a, b)</code>

Construção	Código Python gerado
proclame E	print(E)
confesse \$x	int(input()), float(input()), input() ou teste de pertinência para dogma, conforme o tipo
variável sombreada ou colisão de nomes	o nome recebe o sufixo __N, pois o Python não tem escopo de bloco

Fonte: elaborado pelos autores (2026).

Algumas decisões merecem destaque. A divisão entre dois capítulos emite `//`, porque o `/` do Python sempre devolve *float*; em contrapartida, `//` arredonda para baixo e o C trunca, de modo que o resultado difere apenas com operandos negativos. O *confesse* sobre dogma aceita as respostas `verdade`, `true`, `1` e `sim`. Por fim, o gerador pode anexar comentários `# linha N` ao código, recurso desativado por padrão para manter a saída limpa. As Listagens 1 e 2 mostram, respectivamente, um programa em CatHolyc e o código Python produzido; as Listagens 3 e 4 mostram o mesmo para funções.

Listagem 1 – Programa em CatHolyc (caso.holy)

```
em_nome_do_pai
mandamento capítulo $maioridade seja 18 amém
preceito capítulo $idade seja 21 amém
preceito dogma $estudante seja verdade amém
caso $idade exaltado_sobre $maioridade em_comunhão_com $estudante
conforme_a verdade
proclame "É maior de idade e estudante." amém
doutra_sorte
proclame "A condição não foi satisfeita." amém
amém_senhor
assim_seja_em_seu_nome_amem
```

Fonte: elaborado pelos autores (2026).

Listagem 2 – Código Python gerado a partir da Listagem 1

```
# em_nome_do_pai
MAIORIDADE = 18
idade = 21
estudante = True
if idade > MAIORIDADE and estudante == True:
    print("É maior de idade e estudante.")
else:
    print("A condição não foi satisfeita.")
```

Fonte: elaborado pelos autores (2026).

Listagem 3 – Funções em CatHolyc (funcao_soma.holy)

```
em_nome_do_pai
oficio capítulo $soma capítulo $a capítulo $b
retorne $a somado_a $b amém
```

```

amém_senhor
ofício capítulo $fatorial capítulo $n
    preceito capítulo $r seja 1 amém
    preceito capítulo $i seja 1 amém
    enquanto $i não_acima_de $n
        $r seja $r multiplicado_por $i amém
        $i seja $i somado_a 1 amém
    amém_senhor
    retorne $r amém
amém_senhor
proclame $soma(2, 3) amém
proclame $fatorial(5) amém
assim_seja_em_seu_nome_amem

```

Fonte: elaborado pelos autores (2026).

Listagem 4 – Código Python gerado a partir da Listagem 3

```

# em_nome_do_pai
def soma(a, b):
    return a + b
def fatorial(n):
    r = 1
    i = 1
    while i <= n:
        r = r * i
        i = i + 1
    return r
print(soma(2, 3))
print(fatorial(5))

```

Fonte: elaborado pelos autores (2026).

7 IMPLEMENTAÇÃO E ORGANIZAÇÃO DO CÓDIGO

O transpilador foi implementado em Python 3 (versão 3.10 ou superior), utilizando apenas a biblioteca padrão; o pacote `pytest` é necessário somente para a suíte automatizada. O README do projeto traz um guia de utilização com o primeiro programa e todas as opções da linha de comando. O código-fonte está organizado nos seguintes módulos:

- a) `lexer/` (`TokenType.py`, `Token.py`, `lexer.py`): tipos de token, tabela de palavras reservadas e analisador léxico;
- b) `parser/` (`ast_nodes.py`, `parser.py`): definição dos nós da AST e analisador sintático por descida recursiva;
- c) `semantic/` (`symbols.py`, `analyzer.py`): tabela de símbolos com escopos e verificação de tipos;
- d) `codegen/py_generator.py`: gerador de código Python a partir da AST;

- e) `cli.py`: interface de linha de comando, que encadeia as fases e trata opções e códigos de saída;
- f) `scripts/`: atalhos `dump_tokens.py` e `dump_ast.py`; `editors/micro/syntax/holy.yaml`: realce de sintaxe para o editor `micro`;
- g) `tests/`: testes automatizados e programas de exemplo; `README.md`, `SPEC.md` e `DEFESA.md`: documentação do projeto.

A interface de linha de comando oferece as seguintes opções: `-o` (arquivo de saída), `--dump-tokens`, `--dump-ast`, `--emit-py-only` (imprime o Python sem gravar arquivo), `--run` (executa o programa gerado) e `--strict` (transforma avisos de portabilidade em erros). Exemplos de uso:

```
py cli.py programa.holy          # gera programa.py
py cli.py programa.holy --run    # gera e executa
py cli.py programa.holy --dump-tokens --dump-ast
py -m pytest -q                  # suíte automatizada
```

Erros léxicos, sintáticos e semânticos encerram o programa com código de saída 1, e erros de leitura ou escrita de arquivo, com código 2. Todas as mensagens são emitidas em português, no fluxo de erro padrão.

8 SUÍTE DE TESTES

Os testes ponta a ponta usam sete programas, localizados em `tests/programs`, cada um acompanhado do código Python esperado (`.expected_py`) e da saída esperada (`.expected_stdout`). Em cada caso, o programa é transpilado, o Python gerado é comparado com o esperado, o programa é executado e a saída é comparada com a esperada. O Quadro 4 apresenta os programas e suas saídas. Nos casos em que a precedência é relevante, como `$a somado_a $b multiplicado_por $c`, a árvore garante a ordem correta de avaliação.

Quadro 4 – Programas de teste ponta a ponta e saídas esperadas

Arquivo	O que demonstra	Saída esperada
hello.holy	estrutura mínima de programa e saída	Olá, mundo.

Arquivo	O que demonstra	Saída esperada
vars_const.holy	constante e variáveis dos quatro tipos	21, 1.75, Maria, True e 18 (uma linha cada)
caso.holy	caso, doutra_sorte e conjunção lógica	É maior de idade e estudante.
enquanto_countdown.holy	laço enquanto	3, 2 e 1
peregrine_sum.holy	laço peregrine (soma de 1 a 5)	15
laco_cessai.holy	cessai e perseverai em laço enquanto	1, 3 e 4
funcao_soma.holy	oficio, retorne, chamadas e variáveis locais	5 e 120

Fonte: elaborado pelos autores (2026).

Os erros léxicos, sintáticos e semânticos são verificados pela suíte automatizada de 128 testes, executada por `py -m pytest -q` e organizada por fase do transpilador, conforme o Quadro 5. Cada camada fornece entradas inválidas e confere a mensagem e a posição do erro, como número com ponto final, texto não fechado, `$` isolado, comentário de bloco sem fechamento, `amém` ausente, `amém_senhor` ausente e identificador sem `$`.

Quadro 5 – Suíte automatizada por camada

Camada	Testes	Cobertura
Léxico	26	palavras reservadas, identificadores com <code>\$</code> , maior casamento, números, textos e escapes, vírgula, chamadas, erros e posição (linha e coluna)
Parser	14	cada comando, precedência, parênteses, cadeia de <code>doutra_sorte_caso</code> , <code>cessai</code> , <code>perseverai</code> , funções, chamadas e erros de <code>amém</code> e <code>amém_senhor</code> ausentes
Semântica	27	redeclaração, uso sem declaração, alteração de constante, tipos incompatíveis, condições, operadores, e as regras de laços e funções (<code>cessai</code> fora de laço, <code>retorne</code> fora de função, aridade, escrita em escopo externo)
Gerador de código	23	mapeamento por operador e comando, <code>//</code> e <code>/</code> , <code>peregrine</code> e <code>em_cada</code> , <code>confesse</code> , valor padrão, constantes, <code>break</code> , <code>continue</code> e funções
Ponta a ponta	14	sete programas, comparando o Python gerado e a saída com os valores esperados
Linha de comando	24	cada opção, arquivo de saída, códigos de saída e <code>--strict</code>

Fonte: elaborado pelos autores (2026).

9 DECLARAÇÃO DE USO DE FERRAMENTAS DE INTELIGÊNCIA ARTIFICIAL

Em atendimento à orientação da disciplina sobre transparência no uso de inteligência artificial, declara-se que foi utilizada a ferramenta **Claude**, assistente de inteligência artificial desenvolvido pela Anthropic, acessada pela interface web do serviço, em outubro de 2026. O uso ocorreu como apoio ao desenvolvimento do trabalho, conforme detalhado no Quadro 6.

Quadro 6 – Uso de inteligência artificial por etapa do trabalho

Etapa	Contribuição da ferramenta
Compreensão do enunciado	leitura do documento da disciplina e levantamento dos requisitos, entregáveis e critérios de avaliação
Projeto e especificação	apoio à definição da sintaxe dos elementos não previstos no vocabulário e à redação da especificação léxica e da gramática em EBNF
Desenvolvimento do transpilador	apoio ao desenvolvimento do código, incluindo um protótipo inicial do transpilador, e revisão do código do repositório CatHolyc (analísadores, gerador de código e interface de linha de comando) [GRUPO: detalhar quais módulos foram escritos com auxílio da ferramenta]
Testes	execução e conferência da suíte automatizada e dos programas de exemplo, e verificação do comportamento do transpilador em casos de erro
Documentação e apresentação	redação e atualização deste relatório e dos slides de apresentação

Fonte: elaborado pelos autores (2026).

A ferramenta pode produzir erros e omissões; por isso, todo o material por ela gerado foi tratado como proposta, e não como resultado final. O grupo declara que revisou o conteúdo produzido, executou o transpilador e a suíte de testes, e permanece integralmente responsável pela especificação, pela implementação e pelas informações apresentadas neste relatório. A ferramenta não substitui a autoria do grupo, e nenhum trecho de terceiros foi reproduzido sem a devida indicação nas referências.

10 CONSIDERAÇÕES FINAIS

O trabalho resultou em uma linguagem temática com léxico e gramática formalmente especificados e em um transpilador que realiza as quatro fases exigidas, da análise léxica à geração de código Python, com uma interface de linha de comando que também executa o programa gerado. A separação entre gramática e semântica, a representação da precedência na estrutura da AST e a suíte de testes com casos válidos e inválidos permitem demonstrar o funcionamento de cada etapa de forma isolada. Entre as limitações atuais estão a divisão inteira, que difere do C para operandos negativos, a interrupção da análise semântica no primeiro erro a restrição do `em_cada` a textos e a proibição de escrever, dentro de uma função, em variáveis de escopo externo. Como possibilidades de evolução, citam-se a inclusão de vetores, que daria sentido pleno ao `em_cada` sobre coleções, a concatenação de textos e a recuperação de erros sintáticos e semânticos para relatar mais de um erro por execução.

REFERÊNCIAS

AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2007.

BIRL. **BIRL – Bambam's "It's Show Time" Recursive Language**. Disponível em: <https://birl-language.github.io/>. Acesso em: 5 out. 2026.

MENEZES, Paulo Blauth. **Linguagens formais e autômatos**. 6. ed. Porto Alegre: Bookman, 2011.

UNIVERSIDADE REGIONAL DO NOROESTE DO ESTADO DO RIO GRANDE DO SUL. **Plano de ensino**: Linguagens Formais e Compiladores. Santa Rosa, 2026.